



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ **ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ**

КАФЕДРА **КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ**

НАПРАВЛЕНИЕ ПОДГОТОВКИ **09.04.01 Информатика и вычислительная техника**

МАГИСТЕРСКАЯ ПРОГРАММА **09.04.01/12 Интеллектуальный анализ больших
данных в системах поддержки принятия решений**

ОТЧЕТ

О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

НА ТЕМУ:

Анализ влияния запросов на хранение графовых данных в распределённых хранилищах

Студент группы ИУ6-43М

(Подпись, дата)

В.М. Козлов

(И.О. Фамилия)

Научный руководитель
от МГТУ им. Н.Э. Баумана

(Подпись, дата)

А.Ю. Попов

(И.О. Фамилия)

бла бла

Реферат

не пустой

Содержание

Реферат	3
Введение	6
1 Анализ методов распределения графовых данных	7
1.1 Методы структурной оптимизации распределения. Metis	7
1.1.1 Алгоритмы свёртки	9
1.1.2 Алгоритмы распределения	10
1.1.3 Алгоритмы уточнения	11
1.2 Методы структурной оптимизации распределения с учётом нагрузки ...	12
1.2.1 TAPER.....	13
1.2.2 Schism	16
1.2.3 SWORD.....	19
1.2.4 WAWPart: workload-aware weighted partitioning.....	24
1.2.5 LIRA.....	28
1.3 Методы потокового распределения (online partitioning)	33
1.3.1 Балансировочные	33
1.3.2 Хеширование	33
1.3.3 Streaming Graph Partitioning.....	34
1.3.4 Fennel алгоритм.....	36
2 Конструкторская часть	40
2.1 Постановка задачи распределения вершин графа	40
2.2 Структурная оптимизация	41
2.2.1 Структура проекта и основные зависимости	41
2.2.2 Классы для представления графовых структур (graph.hpp).....	42
2.2.3 Структура класса Storage	46
2.2.4 Класс оптимизатора (optimizer.hpp)	51
2.3 Streaming Graph Partitioning.....	54
2.4 Алгоритм Fennel	55
2.4.1 Объективная функция	56
2.4.2 Инкрементальное вычисление и формула для δg	56
2.4.3 Структура проекта и основные зависимости	57
2.4.4 Классы для представления графовых структур (graph.hpp).....	57
2.4.5 Структура класса FennelStorage	60

Заключение.....	62
Список использованных источников.....	63
Приложение А.....	65

Введение

не пустое

1 Анализ методов распределения графовых данных

Задача распределения графовых данных заключается в разбиении множества вершин графа на несколько непересекающихся частей (шардов) таким образом, чтобы обеспечить эффективное выполнение запросов над данными, если быть точным, запросов поиска пути, в распределённой среде. Основной целью является минимизация стоимости обработки рабочей нагрузки, которая, как правило, выражается через количество межшардовых переходов при выполнении запросов, при одновременном соблюдении ограничений на балансировку размеров партиций.

Для выполнения работы были проанализированы методы из трёх следующих групп:

1. Методы распределения вершин графа по хранилищам исходя только из структуры графа (методы структурной оптимизации распределения).
2. Методы распределения вершин графа по хранилищам исходя из структуры графа и данных о запросах (методы структурной оптимизации распределения с учётом нагрузки).
3. Методы распределения новых вершин в уже распределённый граф (методы потокового распределения).

Основной метрикой качества распределения выбрано количество межшардовых переходов, то есть рёбер найденных путей, которые лежат между вершинами на разных шардах. Такие переходы требуют значительно больше времени, чем переходы внутри шарда, что делает эту метрику близкой по своей сути к времени выполнения запроса, но при этом зависит только от алгоритма распределения, а не от реализаций алгоритмов поиска пути, способа хранения данных и так далее.

1.1 Методы структурной оптимизации распределения. Metis

Основной целью структурной оптимизации является уменьшение числа межшардовых рёбер, то есть рёбер между вершинами на разных шардах, поскольку именно они приводят к межшардовым переходам при выполнении запросов. Чем меньше таких переходов требуется для обхода графа, тем ниже накладные

расходы на взаимодействие между узлами хранения и тем выше общая производительность системы. Если использовать более известные математические термины, то это методы уменьшения разреза.

Методы структурной оптимизации распределения графовых данных основаны на использовании топологических свойств исходного графа и направлены на построение такого разбиения вершин, которое отражает его внутреннюю структуру. Основная идея данных подходов заключается в том, что вершины, связанные большим количеством рёбер или находящиеся в плотных подграфах, должны размещаться в одной партии.

Таким образом, задача структурного разбиения может быть сформулирована как поиск разбиения графа, минимизирующего разрез рёбер при соблюдении ограничений на балансировку размеров партий.

METIS (от METis Tournament Inspired Strategy) [1] - это набор программных библиотек и утилит, разработанных в Университете Миннесоты, для разбиения больших неориентированных графов и разрежения матриц. На данный момент считается эталонным решением структурной оптимизации распределения.

Ключевая идея METIS - многоуровневая парадигма (Multilevel Paradigm). Вместо того чтобы работать с огромным исходным графом напрямую, METIS последовательно его упрощает, находит разбиение для маленького графа и затем интерполирует это разбиение обратно на исходный граф. В силу этой многоуровневой парадигмы в контексте Metis будет удобно рассмотреть различные методы, которые могут применяться на каждом уровне.

Алгоритм состоит из трёх этапов:

1. Схлопывание/Упрощение/Свёртка (англ. Coarsening) - уменьшение графа схлопыванием вершин в граф G_l .

Исходный граф G_0 последовательно преобразуется во всё более мелкие графы $G_1, G_2, \dots, G_l$. На каждом шаге пары смежных вершин "схлопываются/сворачиваются" в одну супервершину.

2. Распределение (англ. Partitioning) - распределение графа G_l . Поскольку граф G_l очень мал, для его разбиения можно использовать даже очень дорогие (вычислительно сложные) алгоритмы.

3. Развёртка (англ. Uncoarsening) - развёртка графа G_l , то есть действие обратное первому этапу. Также происходит улучшение (англ. refinement) - оптимизация разбиения в процессе развёртки. Порядок следующий:

(а) Разбиение, найденное для G_l , проецируется на граф G_{m-1} . Так как каждая вершина в G_{m-1} соответствовала вершине в G_l , разбиение определяется автоматически.

(б) Улучшение, то есть переброс вершин между шардами для получения лучшего распределения.

(с) Первые два пункта повторяются пока не будет получено разбиение для G_0 .

Этап хорошо подвергается параллелизации.

1.1.1 Алгоритмы свёртки

Свёртка в Metis сводится к поиску максимального паросочетания, но не наибольшего, так как это слишком затратное действие. и дальнейшего схлопывания рёбер паросочетания.

В базовом методе рассматриваются 4 алгоритма поиска максимального паросочетания:

1. Случайное (англ. Random Matching - RM) - берётся случайное ребро, затем случайное ребро, несмежное с ним, после чего ребро не связанное с выбранными ранее, и так далее пока таких рёбер не останется. Метод прост и эффективен для локальной задачи, но для уменьшения общей мощности разрезов в дальнейшем подходит плохо.

2. Паросочетание тяжёлых рёбер (англ. Heavy Edge Matching - HEM) - выбор рёбер максимального веса по очереди. Экспериментально было установлено, что это приводит к хорошему уменьшению общей мощности разрезов. В Metis этот алгоритм используется как основной.

3. Паросочетание лёгких рёбер (англ. Light Edge Matching - LEM) - выбор рёбер минимального веса по очереди. Алгоритм приводит к большему общему весу рёбер у свёрнутого графа, что может положительно влиять на качество улучшения некоторыми алгоритмами.

4. Паросочетание тяжёлых клик (англ. Heavy Clique Matching - HCM) - объединяет вершины, максимизируя плотность ребер создаваемого подграфа. В отличие от HEM, который выбирает тяжелые ребра, HCM оценивает, насколько две вершины (точнее вершины, которые свернулись в них) близки к формированию клики. Для вершин u и v вычисляется плотность рёбер (англ.

edge density) по формуле:

$$\text{density}(u, v) = \frac{2(ce(u) + ce(v) + ew(u, v))}{(vw(u) + vw(v))(vw(u) + vw(v) - 1)}$$

, где $ce(u)$, $ce(v)$ - веса уже схлопнутых рёбер в вершинах, $ew(u, v)$ - вес ребра между u и v , $vw(u)$, $vw(v)$ - сумма степеней вершин, схлопнутых в данные.

Алгоритм схож с НЕМ, но учитывает и вершины изначального графа.

1.1.2 Алгоритмы распределения

Этап распределения является решением поставленной выше задачи для свёрнутого графа. В Metis рекурсивно используются алгоритмы бисекции графа, поэтому число шардов k должно быть степенью двойки, другие k приведут к плохой балансировке.

На этом этапе также рассматриваются 4 алгоритма:

1. Спектральная бисекция (англ. Spectral Bisection - SB) - Вычисляет собственный вектор Фидлера y матрицы Лапласиана $Q = D - A$, где:

$$a_{ij} = \begin{cases} ew(v_i, v_j) & \text{если } (v_i, v_j) \in E_m \\ 0 & \text{иначе} \end{cases}$$

$$d_{ii} = \sum_{(v_i, v_j) \in E_m} ew(v_i, v_j)$$

Разбиение: $P[j] = 1$ если $y_j \leq r$, и $P[j] = 2$ в противном случае, где r - выбранная взвешенная медиана y_j .

2. Алгоритм Кернигана-Лина (англ. Kernighan-Lin - KL) - алгоритм, работающий на идее улучшения разбиения путём обмена вершин между шардами. Для этого определена следующая функция улучшения распределения:

$$g_v = \sum_{(v,u) \in E \wedge P[v] \neq P[u]} w(v, u) - \sum_{(v,u) \in E \wedge P[v] = P[u]} w(v, u)$$

при положительном g_v перемещение вершины v приведёт к улучшению распределения. За одну итерацию алгоритм проходится по всем вершинам и пытается переместить все, итерации прерываются как только никакое перемещение не приведёт к улучшению, либо когда будет достигнуто предельное число операций. Эксперименты показали, что для достаточно хорошего распределения как

правило хватает 5-10 итераций.[1]. Для избегания работы "вхолостую" итерация прерывается при 50 перемещений без улучшения, что хорошо показало себя при экспериментах. Также стоит отметить, что ещё можно ограничить алгоритм на минимальное кол-во перемещений за итераций.

Функция улучшения может быть рассчитана для всех вершин в начале итерации и при перемещении вершины обновляться только для соседей перемещённой вершины для оптимизации.

3. Алгоритм роста графа (англ. Graph Growing - GGP) - выбирается случайная вершина и от неё как при поиске в ширину обходятся вершины, формируя подграф, пока не будет набрана ровно половина вершин. Этот подграф будет первым шардом, остальные вторым. Далее рекомендуется применить KL для улучшения, что не займёт много времени в силу малого размера свёрнутого графа. Также стоит отметить, что вполне можно делить не пополам, а на другие равные части и применять KL к шардам попарно.

4. Жадный алгоритм роста графа (англ. Greedy Graph Growing - GGGP) - так как для предыдущего алгоритма и так рекомендуется применять KL после бисекции можно сразу при обходе графа идти по вершинам, которые будут давать наибольшее улучшение.

1.1.3 Алгоритмы уточнения

В Metis предлагаются два схожих алгоритма:

1. Алгоритм уточнения Кернигана-Лина - предлагается продолжать использовать KL на каждом новом развёрнутом графе G_i . Так как G_{i+1} уже был хорошо распределён большого кол-ва перемещений не потребуется. Эксперименты показали, что на первой итерации получается наибольшее улучшение, поэтому применение только одной итерации выделяют как алгоритм KL(1) как значительно менее затратный, чем KL.

2. Граничный алгоритм уточнения Кернигана-Лина (англ. Boundary Kernighan-Lin Refinement - BKL) - так как итерации прерываются большинство вычислений тратятся впустую, особенно в KL(1), поэтому предлагается применять KL только к "граничным" вершинам, то есть к тем, которые смежны с вершинами из другого шарда. Это позволяет сильно уменьшить вычислительные затраты, что в свою очередь позволяет чаще делать несколько итераций, что ведёт к идее алгоритма BKL(*,1) - применять BKL пока вершин немного и

перейти на $BKL(1)$, когда их станет слишком много.

Из всего вышесказанного стоит выделить алгоритм уточнения Кернигана-Лина, и его модификацию — граничный алгоритм уточнения Кернигана-Лина —, позволяющие улучшать уже существующее распределение, если оно было каким-либо образом модифицировано. Больше о его возможном применении дальше.

1.2 Методы структурной оптимизации распределения с учётом нагрузки

Классические методы структурной оптимизации распределения, как правило, ориентированы на минимизацию разреза графа и улучшение локальности хранения на основе его топологической структуры. Такие подходы позволяют эффективно учитывать статические свойства графа, однако они не отражают реальное поведение системы при выполнении запросов. На практике стоимость обработки данных определяется не только структурой графа, но и характером рабочей нагрузки, то есть распределением и частотой запросов к вершинам и рёбрам.

К примеру, такой алгоритм может переместить вершину A с шарда 1 на соседний шард 2, чтобы вместо 10 рёбер из шарда 2 в вершину 1 было 2 ребра из шарда 1 в вершину A . Это минимизирует разрез, но может оказаться, что эти два ребра постоянно частвуют в запросах, а изначальные 10 — нет, таким образом уменьшение разреза не приводит к уменьшению количества межшардовых переходов, что фактически увеличивает время выполнения запросов.

Поэтому возникает необходимость в методах, учитывающих рабочую нагрузку. В отличие от классических структурных подходов, они используют информацию о запросах для построения модели взаимодействий между данными и оптимизируют разбиение непосредственно относительно стоимости выполнения реальной нагрузки.

В данном разделе рассматриваются представители данного класса методов.

1.2.1 TAPER

TAPER (query-aware, partition-enhancement for large, heterogeneous, graphs)[2] — система предложенная Хьюго Фертом и Паоло Миссье. Данное решение представляет собой фреймворк для инкрементального улучшения существующих распределений графов, используя специальную структуру TPSTrie.

Экстраверсия и интроверсия вершин

Для количественной оценки вклада каждой вершины в межшардовые переходы вводятся понятия интроверсии и экстраверсии. Эти показатели базируются на вероятностной модели обхода графа, учитывающей паттерны запросов.

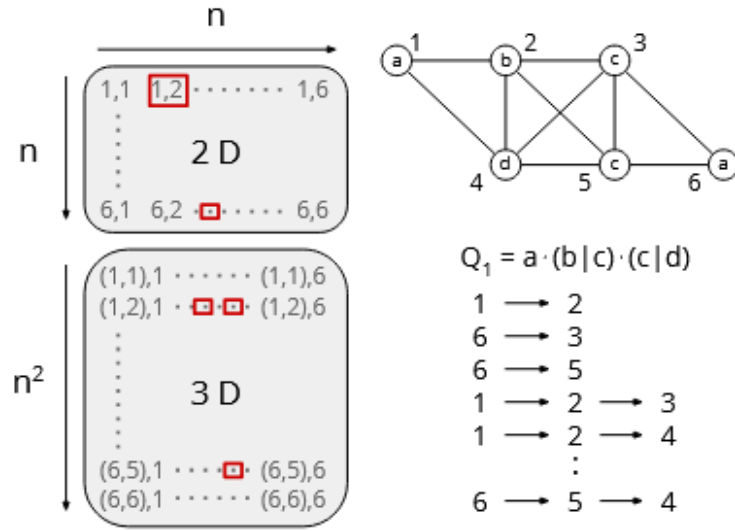


Рис. 1: Структура Visitor Matrix (VM) для хранения вероятностей путей различной длины.

Пусть задан путь $p = v_{i_1} \rightarrow \dots \rightarrow v_{i_k} \rightarrow v$, соответствующий одному из паттернов запросов. Вероятность этого пути $Pr(p)$ может быть вычислена как произведение условных вероятностей переходов. Интроверсия вершины $v \in V_i$ (где V_i — её родной шард) определяется как суммарная вероятность всех путей, ведущих к v , при условии, что следующий шаг также остаётся внутри V_i :

$$\text{introversion}(v) = \frac{1}{Pr(v)} \sum_{p \in \text{paths}(v, V_i)} \left(Pr(p) \cdot \sum_{v' \in V_i} VM_i(t)[p, v'] \right),$$

где $VM_i(t)$ — матрица посещений (Visitor Matrix) для шарда i , хранящая вероятности переходов для путей длины до t , а $Pr(v)$ — нормировочный коэффициент (суммарная вероятность всех путей, ведущих к v). Экстраверсия, соответствен-

но, является дополнением до единицы:

$$\text{extroversion}(v) = 1 - \text{introversion}(v).$$

Именно экстраверсия служит для TAPER сигналом к перемещению вершины.

Представление нагрузки запросов: TPSTrie

Прямое вычисление VM имеет экспоненциальную сложность $O(|V|^t)$ и неприменимо для больших графов. Для решения этой проблемы предлагается компактная структура данных — префиксное дерево обобщения паттернов обхода (Traversal Pattern Summary Trie, TPSTrie). Идея заключается в том, чтобы хранить не вероятности для конкретных вершин, а вероятности для последовательностей меток, которые требуются в запросах.

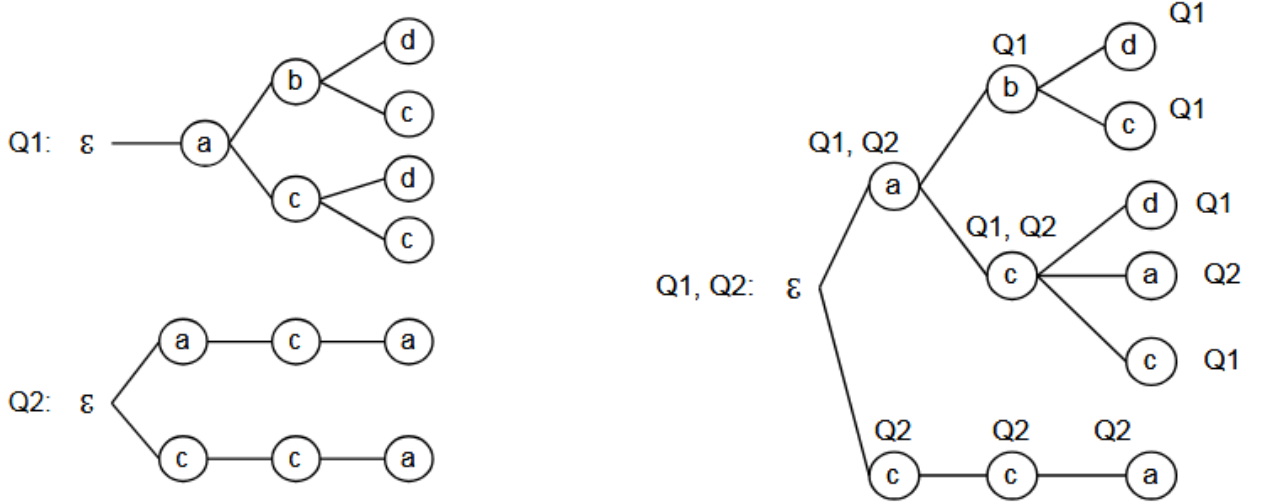


Рис. 2: Построение TPSTrie путём разбора регулярных выражений запросов и объединения результатов.

Каждый узел TPSTrie соответствует некоторой последовательности меток и помечается множеством запросов, которые могут породить такую последовательность. Каждому узлу ставится в соответствие вероятность $p(n)$, вычисляемая на основе частот запросов и структуры дерева. Например, для корневого перехода $\mathcal{E} \rightarrow a$ вероятность равна:

$$Pr(\mathcal{E} \rightarrow a) = \sum_{Q_i \in \mathcal{Q}} Pr(\mathcal{E} \rightarrow a | Q_i) \cdot Pr(Q_i),$$

где условные вероятности определяются количеством возможных первых шагов в запросе Q_i . При изменении частот запросов вероятности в узлах дерева

пересчитываются, что позволяет TPSTrie адаптироваться к динамике нагрузки.

От TPSTrie к Visitor Matrix

Для получения вероятностей переходов между конкретными вершинами (т.е. заполнения VM) используется следующий подход. Для текущей вершины v_k и истории пути p выполняется поиск соответствующего узла в TPSTrie. Полученный узел указывает, какие метки допустимы для следующего шага, и с какими вероятностями. Затем эти вероятности равномерно распределяются среди всех соседей вершины v_k , имеющих данную метку. Это даёт вектор вероятностей перехода к каждому соседу, который и является строкой VM. Такой подход позволяет избежать хранения вероятностей для всех возможных путей, заменяя их компактным представлением на уровне меток.

Для дальнейшего снижения вычислительной и пространственной сложности авторы вводят эвристики:

- Вершины, у которых нет внешних соседей, автоматически считаются безопасными ($\text{introversion} = 1$) и исключаются из рассмотрения.
- Вычисление экстраверсии вершины прекращается досрочно, если её накопленное значение интроверсии превышает заданный порог безопасности. Это позволяет сосредоточить ресурсы только на наиболее проблемных вершинах.

Алгоритм уточнения распределения

Процесс уточнения распределения в TAPER является итеративным и основан на жадном алгоритме рефининга, схожем с Кернигана-Лина/Феддучи-Маттея (Kernighan-Lin / Fiduccia-Mattheyses), но с использованием экстраверсии в качестве целевой функции. Одна итерация (вызов TAPER) состоит из нескольких внутренних шагов, на каждом из которых выполняются следующие действия:

1. Для каждого шарда строится очередь кандидатов — вершин с наибольшей экстраверсией.
2. Для вершины-кандидата v определяется её семья (family). Это множество вершин, которые с высокой вероятностью участвуют в тех же путях, что и v (т.е. являются источниками путей к v с вероятностью выше 0.5). Перемещение всей семьи вместе с кандидатом повышает эффективность обмена.
3. Для семьи вычисляется предпочтительный целевой шард V_j (как правило, тот, где находится большинство соседей, участвующих в общих путях).

4. Происходит двухсторонний обмен (offer/receive):
 - Исходный шард оценивает потерю интроверсии при удалении семьи.
 - Целевой шард оценивает потенциальный выигрыш в интроверсии от добавления семьи.
 - Если выигрыш целевой шард превышает потерю исходной, обмен считается кооперативным и выполняется.
5. Если обмен отклонён, кандидату предлагаются другие соседние шарды в порядке убывания предпочтительности.

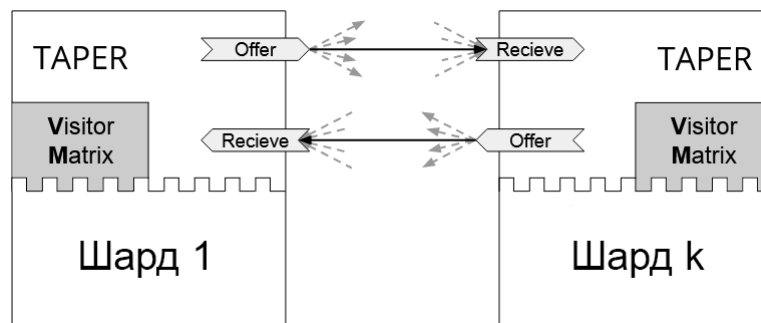


Рис. 3: Схема кооперативного обмена вершинами между шардами.

Каждая вершина может быть перемещена не более одного раза за внутреннюю итерацию. После обработки всех кандидатов запускается следующая внутренняя итерация. Процесс повторяется до тех пор, пока не будет достигнуто целевое улучшение или максимальное число итераций, при этом строго контролируется баланс размеров шардов (допустимое отклонение, например, 5%).

1.2.2 Schism

Schism [3] представляет собой метод разбиения графовых и реляционных структур, ориентированный на минимизацию стоимости выполнения рабочей нагрузки запросов.

Модель рабочей нагрузки

Для построения модели рабочей нагрузки используется журнал запросов Q . На основе этого журнала формируется вспомогательная структура, отражающая частоту совместного доступа к вершинам графа.

Для пары вершин $u, v \in V$ вводится функция совместного использования:

$$w(u, v) = |\{q \in Q \mid u \in q \wedge v \in q\}|.$$

Данная функция определяет количество запросов, в которых одновременно участвуют вершины u и v . Чем выше значение $w(u, v)$, тем более выгодно размещать данные вершины на одном шарде.

На основе данной функции строится взвешенный граф рабочей нагрузки:

$$G_w = (V, E_w)$$

, где E_w включает пары вершин с ненулевым значением $w(u, v)$.

Интерпретация графа рабочей нагрузки

Граф G_w не отражает структуру исходного графа данных, а моделирует поведение пользователей системы. Вершины данного графа соответствуют сущностям исходной базы данных, а рёбра отражают частоту их совместного использования в рамках запросов.

Вес ребра интерпретируется как мера зависимости между данными объектами с точки зрения рабочей нагрузки. Таким образом, задача разбиения сводится к минимизации разрезов именно в этом графе, а не в исходной структуре G .

Построение модели оптимизации

Schism преобразует задачу разбиения в задачу оптимизации, где целевая функция соответствует стоимости выполнения рабочей нагрузки. Для каждого запроса $q \in Q$ определяется количество межшардовых переходов, возникающих при его выполнении.

Пусть запрос q затрагивает множество вершин, распределённых по различным шардам. Тогда стоимость запроса определяется как функция количества переходов между шардами:

$$C(q) = f(\{P_i \mid v \in q, v \in P_i\})$$

, где функция f отражает количество межузловых взаимодействий.

Инициализация разбиения

На начальном этапе алгоритма формируется начальное разбиение $\Pi^{(0)} = \{P_1^{(0)}, \dots, P_k^{(0)}\}$. Данное разбиение может быть получено случайным образом либо с использованием простой эвристики, основанной на локальных свойствах графа.

Качество начального разбиения не является критическим фактором, по-

сколькx дальнейшая оптимизация выполняется итеративно.

Итеративная оптимизация

Основной этап Schism заключается в итеративной локальной оптимизации разбиения. Для каждой вершины $v \in V$ рассматривается возможность её перемещения из текущего шарда P_i на другой P_j .

Изменение принимается, если выполняется условие улучшения:

$$\Delta C < 0$$

, где ΔC обозначает изменение стоимости рабочей нагрузки после перемещения вершины.

Процесс повторяется до достижения состояния, в котором дальнейшие улучшения становятся невозможными или незначительными.

Стоит отметить, что это крайне близко по сути с итерациями алгоритма Кернигана-Лина.

Балансировка шардов

Помимо минимизации стоимости запросов, алгоритм Schism учитывает ограничение на размер шардов:

$$|P_i| \leq (1 + \epsilon) \frac{|V|}{k}.$$

Это ограничение предотвращает возникновение дисбаланса между узлами хранения, который может привести к снижению производительности системы.

При нарушении ограничения выполняются корректирующие перемещения вершин между шардами.

Вычислительная сложность

Основная вычислительная сложность Schism связана с построением графа рабочей нагрузки и оценкой стоимости запросов. При наивной реализации построение G_w требует анализа всех пар вершин, участвующих в запросах, что может приводить к высокой вычислительной нагрузке.

Итеративная оптимизация имеет сложность, зависящую от числа вершин $|V|$, числа шардов k и числа итераций I :

$$O(I \cdot |V| \cdot k).$$

Свойства получаемого разбиения

Получаемое в результате работы Schism разбиение обладает свойством локализации рабочей нагрузки, при котором вершины, часто используемые совместно, стремятся находиться на одном шарде.

Это приводит к снижению числа межузловых обращений при выполнении запросов и повышению эффективности обработки рабочей нагрузки в распределённых системах.

Ограничения метода

Несмотря на эффективность, Schism имеет ряд ограничений. Основным из них является высокая стоимость построения точной модели рабочей нагрузки, требующей анализа большого количества пар взаимодействий между вершинами.

Кроме того, метод предполагает относительно стабильную рабочую нагрузку, так как существенные изменения характера запросов требуют пересчёта разбиения.

Также следует отметить, что при увеличении размера графа и числа запросов возрастает как вычислительная, так и память-затратная сложность построения модели взаимодействий.

Сравнение с классическими методами

В отличие от классических методов edge-cut разбиения, которые минимизируют число разрезанных рёбер исходного графа, Schism минимизирует стоимость выполнения рабочей нагрузки.

Таким образом, если традиционные методы оптимизируют выражение

$$\min |E_{\text{cut}}|$$

, то Schism оптимизирует выражение

$$\min C(Q).$$

Это делает Schism более чувствительным к реальному поведению системы, но одновременно увеличивает сложность построения модели разбиения.

1.2.3 SWORD

Метод SWORD [4] в отличие от ранее рассмотренного метода Schism, который использует детализированную модель совместного доступа между вер-

пинами графа, SWORD ориентирован на снижение вычислительной сложности за счёт использования сжатого представления рабочей нагрузки и инкрементального обновления разбиения.

Ключевое отличие SWORD заключается в том, что он не стремится к построению полной модели взаимодействий между всеми вершинами графа. Вместо этого используется приближённая структура, отражающая наиболее значимые зависимости, возникающие в реальной рабочей нагрузке. Это позволяет существенно повысить масштабируемость метода при сохранении приемлемого качества разбиения.

Общая концепция метода

SWORD основан на идее сокращения пространства анализа рабочей нагрузки. Вместо построения плотного графа совместного доступа, как в Schism, используется разреженное представление взаимодействий между вершинами.

Основная гипотеза метода заключается в том, что для эффективного разбиения графа достаточно учитывать только наиболее значимые связи, возникающие в рабочей нагрузке. Менее частые или случайные взаимодействия могут быть отброшены без существенного ухудшения качества разбиения.

Таким образом, SWORD можно интерпретировать как метод, который заменяет точное моделирование рабочей нагрузки на её структурно сжатую аппроксимацию.

Модель рабочей нагрузки

На первом этапе SWORD строит приближённую модель рабочей нагрузки на основе анализа журнала запросов Q . В отличие от Schism, где рассматриваются все пары совместного использования вершин, здесь применяется агрегирование информации.

Вводится функция значимости взаимодействия между вершинами:

$$w(u, v) \approx f(Q)$$

, где функция $f(Q)$ определяется на основе частоты совместного участия вершин u и v в запросах, однако вычисляется не через полный перебор всех пар, а через агрегированные структуры.

Далее формируется разреженный граф рабочей нагрузки:

$$G_w = (V, E_w)$$

, где множество рёбер E_w определяется условием:

$$E_w = \{(u, v) \mid w(u, v) > \theta\}.$$

Порог θ используется для отсеечения слабых связей, которые не оказывают существенного влияния на стоимость выполнения запросов.

Сжатие и агрегация запросов

Одним из ключевых этапов SWORD является предварительное сжатие рабочей нагрузки. Вместо анализа каждого запроса отдельно выполняется группировка запросов по схожести их структуры.

Пусть множество запросов Q разбивается на кластеры:

$$Q = \bigcup_{i=1}^l Q_i$$

, где каждый кластер Q_i соответствует определённому типу доступа к графу.

В отличие от Schism, где точная статистика запросов используется напрямую, SWORD опирается на агрегированные характеристики кластеров. Для каждого кластера вычисляется обобщённая модель взаимодействий, которая используется для обновления графа G_w .

Такой подход позволяет существенно снизить вычислительные затраты при построении модели рабочей нагрузки, особенно при большом объёме запросов.

Инициализация разбиения

После построения приближённой модели рабочей нагрузки выполняется инициализация разбиения графа. Начальное разбиение $\Pi^{(0)}$ формируется эвристически:

$$\Pi^{(0)} = \{P_1^{(0)}, P_2^{(0)}, \dots, P_k^{(0)}\}.$$

В отличие от Schism, где качество начальной инициализации может существенно влиять на итоговый результат из-за более сложной модели оптимизации, SWORD менее чувствителен к начальному разбиению благодаря использованию локальных итеративных обновлений.

Функция стоимости разбиения

Стоимость выполнения рабочей нагрузки определяется количеством меж-партиционных взаимодействий, возникающих при обработке запросов.

Для запроса q стоимость может быть выражена как:

$$C(q) = g(\{P_i \mid v \in q, v \in P_i\}),$$

где функция g отражает число переходов между партициями при выполнении запроса.

Общая стоимость системы определяется как:

$$C(Q) = \sum_{q \in Q} C(q).$$

В отличие от Schism, где данная функция вычисляется на основе точного графа совместного доступа, в SWORD она является аппроксимацией, зависящей от разреженной модели G_w .

Итеративная локальная оптимизация

Основной этап алгоритма SWORD заключается в итеративной оптимизации разбиения. Для каждой вершины $v \in V$ рассматривается возможность её перемещения между партициями.

Перемещение вершины из P_i в P_j принимается, если выполняется условие:

$$\Delta C < 0.$$

Здесь ΔC обозначает изменение стоимости рабочей нагрузки после переноса вершины.

Процесс оптимизации выполняется локально, без глобального пересчёта всей модели взаимодействий. Это является важным отличием от Schism, где оптимизация тесно связана с точной структурой графа совместного доступа.

Локальный характер обновлений позволяет существенно снизить вычислительную сложность и повысить масштабируемость метода.

Инкрементальное обновление модели

SWORD поддерживает механизм инкрементального обновления рабочей нагрузки. При поступлении новых запросов не требуется полное перестроение графа G_w . Вместо этого обновляются только те части модели, которые затронуты изменениями в рабочей нагрузке.

Это достигается за счёт хранения агрегированных статистик, позволяю-

щих локально корректировать веса рёбер без глобального пересчёта всех взаимодействий.

Данный механизм особенно важен в условиях динамически изменяющейся нагрузки, характерной для реальных графовых приложений.

Балансировка партиций

Как и в большинстве методов разбиения, в SWORD учитывается ограничение на размер партиций:

$$|P_i| \leq (1 + \epsilon) \frac{|V|}{k}.$$

При перемещении вершин между партициями контролируется выполнение данного ограничения. В случае его нарушения выполняются корректирующие операции перераспределения.

Вычислительная сложность

Основное преимущество SWORD заключается в снижении вычислительной сложности по сравнению с Schism. Построение модели рабочей нагрузки осуществляется на основе агрегированных данных, что уменьшает затраты на анализ запросов.

Итеративная оптимизация имеет следующую оценку сложности:

$$O(I \cdot |V| \cdot k),$$

где I обозначает число итераций до сходимости.

При этом ключевая экономия достигается на этапе построения графа рабочей нагрузки, который в SWORD не требует полного перебора всех пар вершин.

Сравнение со Schism

Сопоставляя SWORD с Schism, можно выделить фундаментальное различие в уровне детализации модели рабочей нагрузки. Schism использует точную модель взаимодействий между вершинами графа, основанную на полном анализе запросов, тогда как SWORD применяет приближённую и разреженную модель.

Это приводит к следующему компромиссу: снижение точности моделирования компенсируется значительным увеличением масштабируемости. В результате SWORD оказывается более пригодным для систем с большим объёмом

данных и высокой динамикой рабочей нагрузки.

Кроме того, SWORD лучше адаптируется к изменениям характера запросов благодаря инкрементальному обновлению модели, тогда как Schism требует более глобального пересчёта разбиения при существенных изменениях workload.

Характеристика получаемого разбиения

Получаемое разбиение характеризуется локализацией наиболее частых взаимодействий между вершинами графа. Вероятность межпартиционных обращений снижается за счёт группировки часто совместно используемых данных.

При этом менее значимые взаимодействия могут оставаться разрезанными между партициями, что является следствием использования разреженной модели рабочей нагрузки.

Таким образом, итоговое разбиение представляет собой компромисс между точностью учёта взаимодействий и вычислительной эффективностью.

1.2.4 WAWPart: workload-aware weighted partitioning

Метод WAWPart [5] в отличие от ранее рассмотренных подходов (Schism и SWORD), где основное внимание уделяется либо совместному использованию вершин в запросах, либо сжатому представлению рабочей нагрузки, WAWPart фокусируется на явном моделировании структуры обходов графа и интенсивности переходов между вершинами.

Ключевая идея метода заключается в том, что стоимость распределённого выполнения запросов определяется не только тем, какие вершины используются совместно, но и тем, как именно они последовательно посещаются в процессе обхода графа. Поэтому основным объектом оптимизации становится не просто edge-cut, а weighted traversal-cut, основанный на частоте и структуре переходов между вершинами.

Ключевая идея: переход от вершин к переходам

В отличие от Schism и SWORD, где базовой единицей моделирования является совместное использование вершин в запросах, WAWPart использует переходы между вершинами как основную единицу анализа.

Пусть запрос $q \in Q$ задаёт последовательность:

$$q = (v_1, v_2, \dots, v_n).$$

Тогда он индуцирует множество переходов:

$$T(q) = \{(v_i, v_{i+1}) \mid i = 1, \dots, n - 1\}.$$

Именно эти переходы являются основой построения модели рабочей нагрузки.

Модель рабочей нагрузки

WAWPart строит граф рабочей нагрузки:

$$G_w = (V, E_w)$$

, где E_w отражает интенсивность переходов между вершинами в рабочей нагрузке.

В отличие от классического edge-based представления, здесь каждое ребро $e = (u, v)$ интерпретируется как агрегированная статистика всех переходов $u \rightarrow v$ во всех запросах.

Базовое вычисление веса ребра

Наиболее простая форма веса определяется как количество наблюдаемых переходов:

$$w(u, v) = \sum_{q \in Q} \sum_{(x, y) \in T(q)} \mathbb{I}[x = u \wedge y = v]$$

, где $\mathbb{I}[\cdot]$ — индикаторная функция.

Эта модель отражает абсолютную частоту использования перехода в рабочей нагрузке.

Учёт частоты выполнения запросов

Если различные запросы выполняются с разной частотой, вводится вес запроса $f(q)$:

$$w(u, v) = \sum_{q \in Q} f(q) \cdot \sum_{(x, y) \in T(q)} \mathbb{I}[x = u \wedge y = v].$$

Таким образом, вклад каждого перехода масштабируется в зависимости от реальной интенсивности выполнения запросов.

Это позволяет учитывать не только структуру рабочей нагрузки, но и его распределение во времени.

Нормализация переходов и вероятностная модель

Помимо абсолютных частот, WAWPart использует вероятностную интерпретацию переходов.

Для вершины u вводится вероятность перехода к v :

$$P(u, v) = \frac{N(u, v)}{\sum_{x \in V} N(u, x)}$$

, где $N(u, v)$ — число переходов из u в v .

Тогда вес может быть определён как:

$$w(u, v) = N(u, v) \cdot P(u, v).$$

Такая форма усиливает влияние устойчивых и структурно значимых переходов, снижая влияние случайных шумовых переходов.

Позиционная значимость переходов в обходах графа

Важной особенностью WAWPart является учёт позиции перехода внутри обхода графа.

Пусть переход (v_i, v_{i+1}) находится на позиции i в запросе длины n . Тогда вводится весовой коэффициент:

$$\phi(i, n) = \frac{1}{1 + \alpha \cdot i}$$

, где α — параметр затухания значимости.

Тогда итоговый вклад перехода определяется как:

$$w(u, v) = \sum_{q \in Q} f(q) \cdot \sum_{(v_i, v_{i+1}) \in T(q)} \mathbb{I}[\cdot] \cdot \phi(i, n).$$

Интуитивно это означает, что ранние и критические переходы в обходе могут иметь больший вклад в стоимость разбиения.

Учёт стоимости межпартиционных переходов

WAWPart вводит дополнительную модель стоимости пересечения границ партиций.

Если переход (u, v) выполняется между разными партициями, он вносит штраф:

$$c(u, v) = \delta(u, v) \cdot w(u, v)$$

, где:

$$\delta(u, v) = \begin{cases} 1, & \text{если } u \text{ и } v \text{ находятся в разных партициях,} \\ 0, & \text{иначе.} \end{cases}$$

Таким образом, итоговая цель разбиения сводится к минимизации:

$$\min \sum_{(u,v) \in E_w} w(u, v) \delta(u, v).$$

Пороговая фильтрация рёбер графа рабочей нагрузки

Для уменьшения сложности обработки вводится фильтрация слабых переходов:

$$E_w = \{(u, v) \mid w(u, v) > \theta\}.$$

Это позволяет:

- исключить случайные переходы;
- уменьшить размер графа рабочей нагрузки;
- сфокусироваться на устойчивых структурах обхода графа.

Интерпретация графа рабочей нагрузки

Полученный граф рабочей нагрузки представляет собой не просто структуру совместного использования данных, а ориентированный граф интенсивности переходов при обходах графа.

Вершины соответствуют сущностям данных, а рёбра отражают вероятность и частоту переходов между ними в рамках реальных запросов.

Таким образом, граф рабочей нагрузки в WAWPart является моделью поведения системы, а не только её структуры.

Итоговая формализация

В результате построения графа рабочей нагрузки задача разбиения формулируется как оптимизация weighted traversal-cut:

$$\min \sum_{(u,v) \in E_w} w(u, v) \delta(u, v)$$

, при ограничении балансировки:

$$|P_i| \leq (1 + \varepsilon) \frac{|V|}{k}.$$

Таким образом, WAWPart сводит задачу распределения графовых данных к минимизации стоимости разрезания наиболее интенсивно используемых переходов в структуре обходов графа.

Сравнение со Schism и SWORD

По сравнению с Schism, метод WAWPart использует более специализированную модель рабочей нагрузки, ориентированную на запросы обхода графа. Если Schism моделирует совместное использование вершин в запросах, то WAWPart моделирует непосредственно переходы при обходе графа.

По сравнению с SWORD, WAWPart уделяет больше внимания локальности путей обхода и направлению переходов между вершинами.

Таким образом:

- Schism ориентирован на совместное использование данных в запросах;
- SWORD ориентирован на масштабируемую аппроксимацию рабочей нагрузки;
- WAWPart ориентирован на оптимизацию локальности обходов графа.

1.2.5 LIRA

LIRA (Learning-based Query-aware Partition Framework) — фреймворк, предложенный для решения проблем эффективности приближенного поиска ближайших соседей (ANN). [6]

Задача приближенного поиска ближайших соседей хоть и отлична от поставленной в данной работе задачи, но близка по своей сути, так как в обеих задачах требуется оптимально распределить вершины графа по хранилищам и минимизировать переходы между ними, а решение проблемы длинного хвоста схожа с задачей оптимизации распределения.

Фреймворк, всё же, забегая вперёд, не подходит для решения задачи дипломной, так как применяет популярный в наше время подход, который вкратце можно описать как "А давайте обучим нейросеть и надеяться, что проблема решится а использование нейросети не подходит специфике конечной системы. Однако, чисто алгоритмические составляющие данного фреймворка представляют интерес в контексте решения поставленной задачи.

Проблема длинного хвоста в распределении ближайших соседей

Классические методы ANN, основанные на жестком разбиении данных на шарды (hard partitioning), сталкиваются с фундаментальным ограничением, связанным с природой распределения данных в высокоразмерных пространствах. Даже при удачном выборе центроидов кластеров результаты запроса часто рассеяны по множеству шардов. Это явление, получившее название «длинный хвост» (long-tailed distribution), проявляется в том, что несколько соседей могут находиться на одном шарде, в то время как один-два соседа - в нескольких других.

Авторы провели серию экспериментов на реальных данных SIFT, чтобы подтвердить устойчивость этого феномена. Рисунок 4 демонстрирует, что при разном количестве шардов B (от 8 до 64) тысячи запросов из десяти тысяч имеют распределение, в котором хотя бы один шард содержит ровно одного ближайшего соседа.

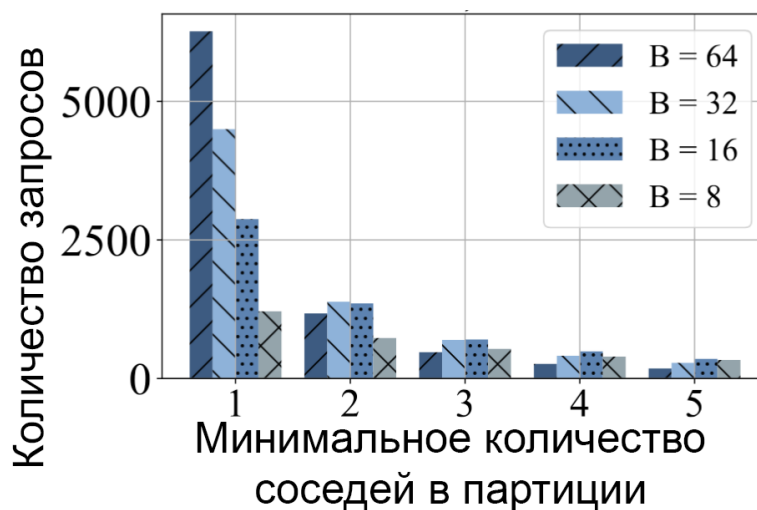


Рис. 4: Распределение запросов с длиннохвостыми kNN. Значительная часть запросов имеет хотя бы один шард с ровно одним соседом.

Этот, казалось бы, незначительный остаток (один вектор в шарде) оказывает непропорционально большое влияние на эффективность поиска. Если не учесть эту редкий шард, полнота поиска упадет. Если учесть - придется тратить ресурсы на просмотр шарда, где находится лишь один полезный результат. Именно для разрешения этой дилеммы и требуется пересмотр самого принципа организации данных.

Концепция редундантности

В ответ на ограничения жесткого разбиения в фреймворке LIRA предлагается отказаться от принципа «одна точка - один шард» и ввести элемент

избыточности (redundancy) в структуру хранения. Ключевая идея заключается в том, чтобы стратегически дублировать определенные вектора данных, тем самым сглаживая длинный хвост распределения и уменьшая количество шардов, которые необходимо посетить для достижения высокой полноты поиска.

Цель редундантности можно проиллюстрировать простым примером. Пусть распределение kNN для некоторого запроса имеет вид $[5, 4, 1, 0, 0]$, что означает необходимость просмотра трех шардов. Если длиннохвостый вектор из третьего шарда продублировать в одну из двух первых, распределение изменится на $[5, 5, 0, 0, 0]$, и для покрытия всех k соседей потребуется просмотреть всего два шарда.

Однако реализация этой идеи сталкивается с двумя серьезными вызовами. Первый вызов связан с идентификацией векторов, которые являются длинным хвостом. Прямой подход требует вычисления kNN для всех точек данных, что имеет квадратичную сложность $O(N^2)$ и неприменим для больших данных. Второй вызов заключается в выборе шарда для дублирования: необходимо определить, в какой именно шард поместить копию вектора, чтобы это максимально сократило будущие запросы.

Алгоритм дублирования

Этап 1: Оценка вероятностей с помощью обученной модели.

Для всех векторов $v \in \mathcal{D}$ вычисляются предсказания обученной модели f , которая для каждого вектора (рассматриваемого как гипотетический запрос) возвращает вектор вероятностей:

$$\hat{p}^v = f(v, I_v) = [\hat{p}_1^v, \hat{p}_2^v, \dots, \hat{p}_B^v] \in [0, 1]^B$$

где $I_v = [dist(v, c_1), dist(v, c_2), \dots, dist(v, c_B)]$ - расстояния от вектора v до центроидов всех шардов, а $\hat{p}_b^v$ интерпретируется как вероятность того, что b -й шард содержит хотя бы одного из k ближайших соседей вектора v .

Этап 2: Оценка собственного пробы для каждого вектора.

Для каждого вектора v вычисляется его собственное прогнозируемое количество шардов, необходимых для покрытия его k ближайших соседей:

$$\hat{n}_v^* = |\{b \mid \hat{p}_b^v > \sigma\}|$$

где σ - пороговое значение (по умолчанию 0.5). Величина $\hat{n}_v^*$ является оценкой того, насколько широко распределены ближайшие соседи самого вектора v по

различным шардам.

Этап 3: Выявление длиннохвостых векторов-кандидатов.

Авторами эмпирически обнаружена корреляция: векторы с большим значением $\hat{n}_v^*$ с высокой вероятностью сами оказываются в длинном хвосте распределения kNN для других запросов. Формально это можно записать как:

$$P(v \in \text{LongTail}(q) \mid \hat{n}_v^* > \tau) \gg P(v \in \text{LongTail}(q))$$

где $\text{LongTail}(q)$ - множество векторов, являющихся единственными представителями своих шардов в результатах запроса q , а τ - некоторый порог.

На основе этой корреляции формируется множество кандидатов на дублирование:

$$\mathcal{D}_{\text{pick}} = \{v \in \mathcal{D} \mid \hat{n}_v^* \geq \text{Percentile}_\eta(\{\hat{n}_u^* \mid u \in \mathcal{D}\})\}$$

где Percentile_η - η -й процентиль распределения $\hat{n}^*$ по всему множеству данных, а η - гиперпараметр алгоритма (обычно 3-5%).

Этап 4: Выбор целевого шарда для дублирования.

Для каждого вектора-кандидата $v \in \mathcal{D}_{\text{pick}}$ необходимо выбрать целевой шард $b_{\text{target}}(v)$, в которую будет помещена его копия. Выбор основывается на следующем эмпирическом наблюдении: множество реплика-шардов $R(v)$ (шардов, в которые следует дублировать вектор для максимального сокращения будущих запросов) с высокой точностью покрывается top-M шардов по значению $\hat{p}_b^v$:

$$\text{Recall}_{\text{rep}}(M) = \frac{|\text{Top}_M(\hat{p}^v) \cap R(v)|}{|R(v)|} \approx 1 \quad \text{при умеренных } M$$

На практике используется следующее правило выбора:

$$b_{\text{target}}(v) = \arg \max_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v$$

то есть выбирается шард с наибольшей предсказанной вероятностью среди всех шардов, кроме исходной.

В случае, если $\arg \max_b \hat{p}_b^v = b_{\text{orig}}(v)$ (что возможно, хотя исходный шард и имеет наибольшую вероятность), выбирается шард со вторым по величине значением $\hat{p}_b^v$:

$$b_{\text{target}}(v) = \arg \max_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v = \arg \max_{b \neq b_{\text{orig}}(v)} \hat{p}_b^v$$

Этап 5: Дублирование и обновление структуры данных.

Для каждого выбранного вектора $v \in \mathcal{D}_{\text{pick}}$ создается его копия v' (полностью идентичный вектор), которая добавляется в целевой шард $P_{b_{\text{target}}(v)}$:

$$P_{b_{\text{target}}(v)} \leftarrow P_{b_{\text{target}}(v)} \cup \{v'\}$$

Исходный вектор v при этом остается в своём шарде $P_{b_{\text{orig}}(v)}$. В результате общий объем данных увеличивается на $|\mathcal{D}_{\text{pick}}|$ векторов, что составляет $\eta\%$ от исходного размера.

Для управления редундантностью используется нейросеть, поэтому дальнейшего погружения в фреймворк в данной работе не будет.

Применение для решения поставленной задачи: решение "проблемы хабов"

Рассмотренный метод Lyra демонстрирует подход к оптимизации распределения данных, основанный на предсказательной модели размещения векторов и учёте вероятностной структуры пространства поиска. Основная идея заключается в использовании обученной модели для выбора целевых шардов, что позволяет существенно снизить количество просматриваемых сегментов при выполнении запросов.

В контексте общей задачи распределения графовых данных особый интерес представляет возможность расширения данного подхода за счёт введения контролируемой редундантности. Дублирование отдельных элементов данных между несколькими шардами может быть использовано как механизм повышения устойчивости и адаптивности системы к нагрузке. В частности, предварительное размещение наиболее “нагруженных” или часто запрашиваемых элементов в нескольких партициях потенциально позволяет снизить стоимость будущих запросов за счёт уменьшения количества межшардовых переходов для вершин-перекрёстков, у которых слишком много проходящих через них путей.

Таким образом, редундантность в рамках подхода Lyra может рассматриваться не как избыточность хранения, а как инструмент оптимизации распределения, позволяющий сместить баланс между стоимостью хранения и стоимостью обработки запросов. Дальнейшее развитие метода может быть связано с формализацией критериев выбора дублируемых элементов и построением стратегии адаптивного перераспределения данных в зависимости от динамики рабочей нагрузки.

1.3 Методы потокового распределения (online partitioning)

В этой секции будут использоваться следующие обозначения: P^t - распределение вершин по шардам (состояние шардов) в момент времени t , $P^t(i)$ - состояние шарда i в момент времени t , v - некая вершина, $N(v)$ - соседи вершины v , C - максимальная вместимость шарда.

1.3.1 Балансировочные

Простейшие методы распределения новых вершин, не решающие задачу, поставленную выше, а предназначенные просто для равномерного распределения данных по шардам. Стоят упоминания, так как используются в последующих методах, а также могут быть использованы на начальных этапах.

Всего их три:

1. Поочерёдное - распределение записей по шардам по очереди. По сути round-robin.
2. Групповое - записи на шарды загружаются сразу большим набором исходя из ожидаемого количества и следующего из этого количества записей на каждом шарде.
3. Случайное/Псевдослучайное - абсолютно или частично случайное распределение вершин по шардам.

1.3.2 Хеширование

Технически попадает в группу методов описанную выше, конкретно в псевдослучайные, но, в отличие от них, активно используется сам по себе, а не как начальный этап или основа для других методов.

Заключается в использовании хеш-функции на каких-либо атрибутах вершин. Для лучшего решения задачи можно хешировать атрибуты, которые естественным образом совпадают у близких вершин, например, географические.

Используется, например, в ArangoDB [7] (Файл `arangod\Sharding\ShardingStrategyDefault.cpp`, метод `getResponsibleShard`), где хеширование применяется дважды: сначала хешируются атрибуты, после чего хешируется полученный хеш. Примечание: это не связано с алгоритмом двойного хеширования для решения коллизий, коллизии в понимании этого алгоритма не случаются при распределении вершин по шардам.

1.3.3 Streaming Graph Partitioning

Потоковое распределение графов [8] (англ. Streaming Graph Partitioning) - алгоритм, а точнее набор алгоритмов, для распределения вершин графа, которые приходят "потоком" по одной или небольшими наборами (оба варианта простейшим образом можно переводить друг в друга, поэтому далее их разделения не будет), с использованием эвристик для оценки насколько новая вершина подходит и штрафов, чтобы не избежать загрузки большинства вершин на один шард.

Метод учитывает три возможных варианта организации потока:

1. Случайное - вершины графа приходят в случайном порядке.
2. Обход в ширину (BFS) - первая вершина выбирается случайно, каждая следующая приходит согласно обходу в ширину.
3. Обход в глубину (DFS) - аналогично обходу в ширину, но согласно обходу в глубину.

Для балансировки распределения в эвристиках используются следующие весовые функции:

1. $w(t, i) = 1$ - без взвешивания.
2. $w(t, i) = 1 - \frac{|P^t(i)|}{C}$ - линейный вес.
3. $w(t, i) = 1 - e^{|P^t(i)-C|}$ - экспоненциальный вес.

Предлагаемые эвристики:

1. Балансировка - распределение по шардам с наименьшим кол-вом вершин.
2. Групповая (Chunking) - описанное выше групповое распределение
3. (Вес) Детерминированная жадная (DG):

$$ind = \operatorname{argmax}_{i \in k} \{|P^t(i) \cap N(v)|w(t, i)\}$$

где $w(t, i)$ - одна из описанных выше весовых функций.

4. (Вес) Случайная жадная (RG): выбор шарда происходит следующего распределения

$$Pr(i) = \frac{|P^t(i) \cap N(v)|w(t, i)}{Z}$$

где $w(t, i)$ - одна из описанных выше весовых функций, а Z - нормализующая константа. В оригинальной статье отмечается, что случайная эвристика рассматривается только потому что она может лучше работать в худшем случае

организации потока.

5. (Вес) Треугольники:

$$ind = \operatorname{argmax}_{i \in k} \left\{ \frac{|E(P^t(u) \cap N(v), (P^t(u) \cap N(v)))|}{\binom{|P^t(u) \cap N(v)|}{2}} \right\} w(t, i)$$

где $w(t, i)$ - одна из описанных выше весовых функций, $E(S, T)$ - набор рёбер между наборами верши S и T . Идея заключается в сохранении наибольшего кол-ва треугольников на шарде, что, очевидно, приведёт к хорошей связности на нём.

6. Балансировка большого При возможности определять степень вершины и насколько она велика относительно всего графа распределять вершины с большой степенью балансировкой, а с маленькой DG.

Дальнейшие эвристики предполагают буферизацию:

7. Предпочтение большому - начать с вершин больших степеней, использовать балансировку большого.

8. Избегание большого - распределить жадным алгоритмом вершины малых степеней, когда останутся только вершины с большими степенями распределить их DG.

9. Жадный EvoCut - использовать EvoCut [9] на подграфе в буфере, найти малые множества (англ. nibbles) с хорошей проводимостью и распределить их по шардам DG.

Метод в первую очередь направлен на графы соц.сетей, где и используется.

Сравнение эвристик

Для оценки качества определения считается процент среднего улучшения по следующей формуле:

$$\text{gain} = \frac{\text{edges_cut_random} - \text{edges_cut_heuristic}}{\text{edges_cut_random} - \text{edges_cut_METIS}} \times 100\%$$

Где edges_cut_random - кол-во ребёр между шардами при хешировании, $\text{edges_cut_heuristic}$ - кол-во рёбер между шардами в эксперименте, а edges_cut_METIS количество рёбер между шардами при использовании METIS.

Метод и его производный Fennel [10] используются в соц. сетях и рекомендательных системах, на графах которых и будет проводиться сравнение.

Сравнение эвристик представлено в следующей таблице:

Сокращение	Эвристика	<i>BFS</i>	<i>DFS</i>	<i>Случайное</i>
В	Балансировка	-1.5	-1.3	-0.2
С	Групповое	37.6	35.7	0.7
Н	Хеширование	-1.9	-2.1	-1.7
DG	Детерминированная жадная	57.7	54.7	45.4
RG	Случайная жадная	45.5	44.9	38.7
Т	Треугольники	49.7	48.4	40.2
LDG	Лин. Дет. жадная	76.0	73.0	75.3
LRG	Лин. Случ. жадная	46.4	44.9	39.1
LT	Лин. Треугольники	55.4	54.6	49.3
EDG	Эксп. Дет. жадная	59.4	56.2	47.5
ERG	Эксп. Случ. жадная	45.6	45.6	38.8
ЕТ	Эксп. Треугольники	50.7	49.3	41.6
ВВ	Балансировка большого	67.8	68.5	63.3
PB	Предпочтение большому	-9.5	-18.6	-23.1
AB	Избегание большого	-27.3	-38.6	-46.4
GE	Жадный EvoCut	60.3	58.6	43.1

Таблица 1: Среднее улучшение каждой эвристики по всем наборам данных и размерам партиций [8]

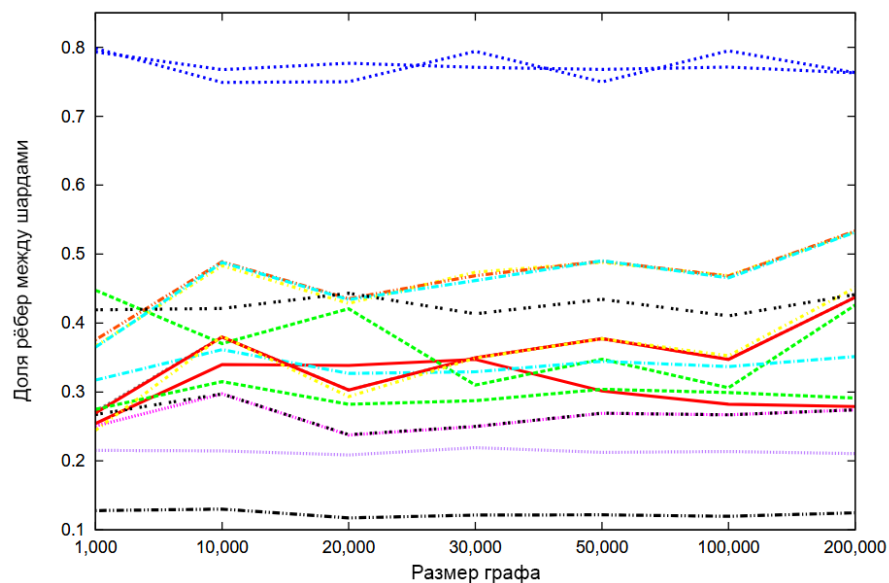


Рис. 5: Визуализация доли рёбер между шардами для 4 шардов с порядком BFS. Нижняя линия - METIS, предпоследняя нижняя - Лин. Дет. жадная [8]

Как можно увидеть по таблице и графику выше LDG даёт наилучшие результаты.

1.3.4 Fennel алгоритм

Fennel алгоритм [10] - модификация потокового распределения графов.

Модификация заключается в определении так называемой объективной функции k -распределения $g(P)$, значение которой должно максимально увеличиться при распределении новой вершины. По своей сути функция является эвристикой метода потокового распределения графов.

Опуская детали вывода функции описанные в оригинальной статье объективная функция k -распределения имеет следующий вид:

$$g(P) = \sum_{i=1}^k [e(S_i, S_i) - p \binom{|S_i|}{2}]$$

где $p = 2\alpha = m \frac{k^{\gamma-1}}{n^{\gamma}}$, $1 \leq \gamma \leq 2$.

Несмотря на относительную громоздкость самой функции, расчёт всех её вариантов не требуется, так как нужно найти максимальное $\delta g(v, S_i) = g(S_1, S_2, \dots, S_i \cup v, \dots, S_k) - g(S_1, S_2, \dots, S_k)$ для $i \in \{1 \dots k\}$, притом что все $g(S_1, S_2, \dots, S_k)$ и $e(S_j, S_j) - p \binom{|S_j|}{2}$ были найдены при заносе предыдущих вершин. Также, очевидно, каждое $g(S_1, S_2, \dots, S_i \cup v, \dots, S_k)$ может быть вычислено на соответствующем шарде, если он способен на вычисления, что позволяет распараллелить все расчёты.

Балансировка

Стоит отдельно упомянуть балансировку, так как алгоритм в "сыром" виде совсем не гарантирует хорошую балансировку, а при $\gamma = 1$, когда алгоритм сводится к отправлению вершин в шард с наибольшим числом её соседей, вообще имеет тенденцию к плохой балансировке.

Поэтому также предложена надстройка, позволяющая гарантировать балансировку для $\rho = \tau$. Она заключается в изначальном отметании шардов, добавление вершины на которую превысит эту нагрузку, то есть таких шардов, где $|S_i \cup v| > \tau \frac{n}{k}$.

Сравнение с другими методами по качеству

Ниже предоставлены сравнения результатов работы алгоритма с другими эвристиками при $\gamma = \frac{3}{2}$ и $\tau = 1.1$.

	BFS		Random	
Метод	λ	ρ	λ	ρ
H	96.9%	1.01	96.9%	1.01
B	97.3%	1.00	96.8%	1.00
LDG	34%	1.01	40%	1.00
EDG	39%	1.04	48%	1.01
T	61%	2.11	78%	1.01
LT	63%	1.23	78%	1.10
ET	64%	1.05	79%	1.01
Fennel	14%	1.10	14%	1.02
METIS	8%	1.00	8%	1.02

Таблица 2: Качество работы различных методов на графе amazon0312 ($k = 32$) [10]

m	k	Fennel		METIS	
		λ	ρ	λ	ρ
7 185 314	4	62.5%	1.04	65.2%	1.02
6 714 510	8	82.2%	1.04	81.5%	1.02
6 483 201	16	92.9%	1.01	92.2%	1.02
6 364 819	32	96.3%	1.00	96.2%	1.02
6 308 013	64	98.2%	1.01	97.9%	1.02
6 279 566	128	98.4%	1.02	98.8%	1.02

Таблица 3: Результаты Fennel и METIS, усреднённые по 5 случайным графам, сгенерированным по модели HP(5 000, k , 0.8, 0.5) [10].

Сравнение с другими методами по времени

	Fennel		LDG		METIS	
Кластеров (k)	λ	ρ	λ	ρ	λ	ρ
2	6.8%	1.1	34.3%	1.04	11.98%	1.02
4	29%	1.1	55.0%	1.07	24.39%	1.03
8	48%	1.1	66.4%	1.10	35.96%	1.03

Таблица 4: Результаты работы методов при распределении графа Twitter (≈ 1.5 млрд рёбер). Распределение через Fennel и LDG заняло [10].

	Время выполнения [с]		Коммуникация [МБ]	
Кластеров (k)	Hash	Fennel	Hash	Fennel
4	32.27	25.49	321.41	196.9
8	17.26	15.14	285.35	180.02
16	10.64	9.05	222.28	148.67

Таблица 5: Средняя длительность шага и средний объем данных, передаваемых за шаг при выполнении [10]

2 Конструкторская часть

2.1 Постановка задачи распределения вершин графа

В контексте распределенных графовых баз данных задача распределения вершин (графового партиционирования) формулируется следующим образом. Пусть дан неориентированный граф $G = (V, E)$, где V - множество вершин, $|V| = n$, а E - множество рёбер, $|E| = m$. Требуется найти такое разбиение (распределение) $P = \{S_1, S_2, \dots, S_k\}$ множества вершин V на k непересекающихся подмножеств (шардов) S_i , что:

- $\bigcup_{i=1}^k S_i = V$.
- $S_i \cap S_j = \emptyset$ для всех $i \neq j$.

Качество распределения оценивается по двум основным критериям, которые необходимо оптимизировать:

1. **Минимизация разрезов (Edge Cut).** Мощность разреза $\partial e(P)$ определяется как количество рёбер, концы которых оказались в разных шардах. Формально:

$$|\partial e(P)| = \sum_{i=1}^k |e(S_i, V \setminus S_i)|$$

Минимизация этого параметра напрямую влияет на снижение сетевого трафика при выполнении распределенных запросов.

2. **Балансировка нагрузки (Load Balancing).** Неравномерное распределение вершин приводит к перегрузке одних узлов и простоям других. Для количественной оценки используется показатель нормализованной максимальной нагрузки ρ , который необходимо минимизировать:

$$\rho = \frac{\max_{i=1}^k (|S_i|)}{n/k}$$

Идеально сбалансированное распределение соответствует $\rho = 1$.

Таким образом, целевая задача заключается в поиске такого распределения P^* , которое минимизирует мощность разрезов $|\partial e(P)|$ при максимально допустимом значении дисбаланса $\rho \leq \tau$, где τ - заданный порог.

2.2 Структурная оптимизация

В рамках данной курсовой работы была реализована вычислительная модель для расчёта и оптимизации метрики улучшения распределения g_v для граничных вершин графа согласно алгоритму Кернигана-Лина. Реализация включает в себя четыре основных компонента:

1. Классы для представления графовых структур (вершины, рёбра, ключи).
2. Классы имитации шины для общения компонентов
3. Класс хранилища (Storage) для управления вершинами и их связями.
4. Класс оптимизатора (StorageOptimizer) для расчёта метрики g_v .

Основной задачей практической части являлось создание инфраструктуры для вычисления функции улучшения распределения, определенной в алгоритме Кернигана-Лина:

$$g_v = \sum_{\substack{(v,u) \in E \\ P[v] \neq P[u]}} w(v, u) - \sum_{\substack{(v,u) \in E \\ P[v] = P[u]}} w(v, u)$$

2.2.1 Структура проекта и основные зависимости

Языком разработки был выбран C++ в силу его низкоуровневости и скорости, а также с намерением в дальнейшем внедрить эти наработки в графовую БД на C++ научного руководителя.

Проект организован в виде набора заголовочных файлов (header files), что соответствует современным подходам разработки на C++. Основные файлы проекта:

- `graph.hpp` – содержит базовые классы для представления графовых структур.
- `interface_bus.hpp` – содержит интерфейс, описывающий основные сообщения в шине.
- `bus.hpp` – содержит простую реализацию имитации шины.
- `storage.hpp` – реализует класс хранилища для управления вершинами.
- `optimizer.hpp` – содержит реализацию оптимизатора с вычислением метрики g_v .
- `main.cpp` – демонстрационный файл с тестовым сценарием.

Ниже представлен релевантный для решения задачи практики код.

2.2.2 Классы для представления графовых структур (graph.hpp)

Класс NodeKey

Класс NodeKey представляет собой обёртку для ключа вершины графа. Он обеспечивает типобезопасность и возможность использования различных типов данных в качестве ключей (целые числа, строки и т.д.).

Листинг 1: Класс NodeKey в graph.hpp

```
1 template <typename KeyType>
2 class NodeKey {
3 public:
4     KeyType key_value;
5
6     NodeKey(): key_value(KeyType()) {};
7     NodeKey(const KeyType& key): key_value(key) {};
8
9     // Оператор присваивания
10    NodeKey<KeyType>& operator=(const NodeKey<KeyType>& other) {
11        if (this != &other) {
12            key_value = other.key_value;
13        }
14        return *this;
15    };
16
17    // Дружественные операторы сравнения
18    friend bool operator<(const NodeKey<KeyType>& lhs, const
19        NodeKey<KeyType>& rhs) {
20        return lhs.key_value < rhs.key_value;
21    }
22
23    friend bool operator>(const NodeKey<KeyType>& lhs, const
24        NodeKey<KeyType>& rhs) {
25        return rhs < lhs;
26    }
27
28    friend bool operator==(const NodeKey<KeyType>& lhs, const
29        NodeKey<KeyType>& rhs) {
30        return lhs.key_value == rhs.key_value;
31    }
32}
```

29

```
30 friend bool operator!=(const NodeKey<KeyType>& lhs , const
    NodeKey<KeyType>& rhs) {
31     return !(lhs == rhs);
32 }
33 };
```

Класс `NodeKey` является шаблонным, что позволяет использовать различные типы данных в качестве ключей вершин. Это важно для обеспечения гибкости при работе с различными типами графовых данных. Известно, что в конечной реализации используются строковые ключи, но была добавлена гибкость для потенциального использования пользовательских гибридных ключей.

Класс `Edge`

Класс `Edge` представляет ребро графа с весом и дополнительными параметрами.

Листинг 2: Класс `Edge` в `graph.hpp`

```
1 template <typename KeyType>
2 class Edge {
3 private:
4     float weight;
5     bool directional;
6     std::map<std::string , Parameter> parameters;
7     NodeKey<KeyType> from;
8     NodeKey<KeyType> to;
9 public:
10    float get_weight() const {
11        return weight;
12    }
13    bool is_directional() const {
14        return directional;
15    }
16    const NodeKey<KeyType>& get_from() const {
17        return from;
18    }
19    const NodeKey<KeyType>& get_to() const {
20        return to;
21    }
22
23    NodeKey<KeyType> get_other(const NodeKey<KeyType>& node)
const {
```

```

24         if (node == to) {
25             return from;
26         } else if (node == from) {
27             return to;
28         }
29
30         return node;
31     }
32
33     NodeKey<KeyType> get_other(NodeKey<KeyType>* node) const {
34         if (*node == to) {
35             return from;
36         } else if (*node == from) {
37             return to;
38         } else {
39             return *node;
40         }
41     }
42 };

```

Класс Node

Класс Node представляет вершину графа и содержит всю информацию о её связях с другими вершинами.

Листинг 3: Класс Node в graph.hpp (часть 1)

```

1 template <typename KeyType>
2 class Node {
3 private:
4     NodeKey<KeyType> key;
5 public:
6     std::map<std::string, Parameter> parameters;
7     std::map<NodeKey<KeyType>, Edge<KeyType>> edges;
8 }
9     NodeKey<KeyType> get_key() const {
10         return key;
11     }
12
13     void add_edge(Edge<KeyType> new_edge) {
14         NodeKey<KeyType> current_key = this->get_key();
15         edges[new_edge.get_other(current_key)] = new_edge;
16     }
17

```

```

18     void add_edges( std::map<NodeKey<KeyType>, Edge<KeyType>>
new_edges) {
19         edges.merge(new_edges);
20     }
21
22     bool remove_edge_to(NodeKey<KeyType> neighbour_key) {
23         return edges.erase(neighbour_key);
24     }
25
26     bool remove_edge(Edge<KeyType> removing_edge) {
27         NodeKey<KeyType> current_key = this->get_key();
28         return edges.erase(removing_edge.get_other(current_key));
29     }
30 };

```

Класс имеет несколько конструкторов для удобного создания вершин с различными конфигурациями связей. Все рёбра хранятся в вершине, от ссылок было решено отказаться, так как в конечной реализации хранилища должны находиться на разных машинах, а в вершины периодически перемещаться между ними.

Класс интерфейса шины (interface_bus.hpp)

Крайне важный интерфейс, через который происходит взаимодействие всей системы. Описывает методы добавления, удаления и запроса вершин, а также объявлений о добавлении и удалении для возможности другим хранилищам знать где находится другая вершина.

И наконец ключевые для алгоритма Кернигана-Лина методы получения граничных вершин и рёбер `ask_neighbours_to_storage` и `ask_edges_to_storage`.

Листинг 4: Базовая структура класса IBus

```

1 template <typename KeyType>
2 class IBus {
3 public:
4     virtual Node<KeyType> request_node(const NodeKey<KeyType>&
node) = 0;
5     virtual int send_add_node(const Node<KeyType>& node) = 0;
6     virtual bool send_add_node(const Node<KeyType>& node, int
storage_id) = 0;
7     virtual bool send_remove_node(const NodeKey<KeyType>& node) =
0;
8     virtual bool send_remove_node(const NodeKey<KeyType>& node,

```

```

    int storage_id) = 0;
9   virtual int ask_who_has(int asker_id, NodeKey<KeyType> key) =
    0;
10  virtual void announce_add(NodeKey<KeyType> key, int
    storage_id, std::set<Edge<KeyType>> edges) = 0;
11  virtual void announce_remove(NodeKey<KeyType> key, int
    storage_id) = 0;
12  // запрашивает у source вершины, соседствующие с target
13  virtual std::set<Node<KeyType>> ask_neighbours_to_storage(int
    source, int target) = 0;
14  // запрашивает у source рёбра, идущие в target
15  virtual std::set<Edge<KeyType>> ask_edges_to_storage(int
    source, int target) = 0;
16 };

```

Класс шины SimpleBus (bus.hpp)

Простая синхронная однопоточная реализация методов IBus. Релевантный код слишком длинный для вставки в основную часть, поэтому представлен в приложении А.

Класс хранилища (storage.hpp)

Класс Storage представляет собой хранилище вершин графа и реализует логику управления внутренними и внешними связями.

2.2.3 Структура класса Storage

Листинг 5: Базовая структура класса Storage

```

1 template <typename KeyType>
2 class Storage {
3 private:
4 typedef Node<KeyType> StorageNode;
5 typedef NodeKey<KeyType> Key;
6 int storage_id;
7 std::map<Key, StorageNode> nodes;
8 // external_edges[storage edge go to][external node][local node] = edge
9 std::map<int, std::map<Key, std::map<Key, Edge<KeyType>>>>
    external_edges;
10 IBus<KeyType> *bus = nullptr;
11
12 public:
13 Storage(int id)

```

```

14     :storage_id(id) {}
15 Storage(int id, std::map<Key, StorageNode> _nodes)
16     :storage_id(id), nodes(_nodes) {}
17
18 int get_id() const { return storage_id; };
19 void connect_to_bus(IBus<KeyType>* _bus) { bus = _bus; };

```

Хранилище идентифицируется уникальным `storage_id` и содержит вершины в виде хэш-таблицы для обеспечения быстрого доступа по ключу.

Добавление вершин с автоматическим созданием связей

Листинг 6: Методы добавления вершин класса Storage

```

1 std::optional<std::set<Edge<KeyType>>> add_node(const
    StorageNode& node){
2     Key key = node.get_key();
3     std::cout << storage_id << " adding " << key.key_value <<
    std::endl;
4     typename std::map<Key, StorageNode>::iterator it =
    nodes.find(key);
5     if (it != nodes.end()) {
6         return std::nullopt;
7     }
8     nodes[key] = node;
9
10    std::set<Edge<KeyType>> external_edges_to_announce;
11    for (typename std::map<Key, Edge<KeyType>>::const_iterator
    edge_it = node.edges.begin(); edge_it != node.edges.end();
    ++edge_it) {
12        const Key& neighbor_key = edge_it->first;
13        const Edge<KeyType>& edge = edge_it->second;
14        std::cout << storage_id << " looks for " <<
    neighbor_key.key_value << std::endl;
15        // Ищем соседа в текущем хранилище
16        typename std::map<Key, StorageNode>::iterator it2 =
    nodes.find(neighbor_key);
17        if (it2 != nodes.end()) {
18            std::cout << storage_id << " node " <<
    neighbor_key.key_value << " is inside, no asking" <<
    std::endl;
19            // Сосед найден в этом же хранилище - добавляем обратное ребро
20            it2->second.add_edge(edge);
21        } else {

```

```

22         std::cout << storage_id << " node " <<
neighbor_key.key_value << " is outside , asking" << std::endl;
23         // Сосед не найден - спрашиваем у шины, где он находится
24         int neighbours_storage_id =
bus->ask_who_has(storage_id , neighbor_key);
25         std::cout << storage_id << " asked for " <<
neighbor_key.key_value << " answer: " << neighbours_storage_id
<< std::endl;
26         if (neighbours_storage_id != -1) {
27             // Сохраняем внешнее ребро
28
external_edges[neighbours_storage_id][neighbor_key][key] =
edge;
29             external_edges_to_announce.insert(edge);
30         }
31         // Если сосед нигде не найден - игнорируем (возможно, будет добавлен
позже)
32     }
33 }
34
35     return external_edges_to_announce;
36 };
37
38 bool add_node_and_announce(const StorageNode& node) {
39     if (bus == nullptr) {
40         return false;
41     };
42     std::optional<std::set<Edge<KeyType>>> external_edges =
add_node(node);
43     if (!external_edges.has_value()) {
44         return false;
45     }
46     bus->announce_add(node.get_key() , storage_id ,
external_edges.value());
47     return true;
48 };

```

Метод `add_node` не только добавляет вершину в хранилище, но и автоматически создает связи с её соседями, что обеспечивает целостность графовой структуры. Это требуется для будущей реализации потокового распределения. Также метод `add_node_and_announce` позволяет при добавлении объявить о до-

бавлении новой вершины.

Удаление вершин с автоматическим удалением связей

Листинг 7: Метод удаления вершин класса Storage

```
1 bool remove_node(const Key& key) {
2     if (!has_node(key)) {
3         return false;
4     };
5     StorageNode node = nodes.find(key)->second;
6     nodes.erase(key);
7
8     for (typename std::map<NodeKey<KeyType>,
9         Edge<KeyType>>::iterator edge_it = node.edges.begin(); edge_it
10        != node.edges.end(); ++edge_it) {
11         Key other_key = edge_it->second.get_other(key);
12         typename std::map<Key, StorageNode>::iterator it =
13         nodes.find(other_key);
14         if (it != nodes.end()) {
15             it->second.remove_edge_to(key);
16         }
17     }
18
19     return true;
20 };
21
22 bool remove_node_and_announce(const Key& key) {
23     if (remove_node(key)) {
24         bus->announce_remove(key, storage_id);
25         return true;
26     }
27     return false;
28 };
```

Методы `remove_node` и `remove_node_and_announce` аналогично их `add` версиям автоматически следят за целостностью графа как локально, так и внешне. Это значительно снижает вычислительную сложность, так как исключает из рассмотрения вершины, не имеющие внешних связей.

Методы для работы с граничными вершинами

Для реализации граничного алгоритма Кернигана-Лина требуется уметь получать вершины, граничащие с другим хранилищем, чем занимается метод

get_nodes_with_neighbors_in_storage используя карту внешних соседей.

Листинг 8: Метод для получения граничных вершин

```
1  std::set<StorageNode> result;
2
3  typename std::map<int, std::map<Key, std::map<Key,
   Edge<KeyType>>>>::const_iterator storage_it =
   external_edges.find(target_storage_id);
4  if (storage_it == external_edges.end()) {
5      return result;
6  }
7
8  const std::map<Key, std::map<Key, Edge<KeyType>>>>& node_edges
   = storage_it->second;
9  for (typename std::map<Key, std::map<Key,
   Edge<KeyType>>>>::const_iterator node_edges_it =
   node_edges.begin(); node_edges_it != node_edges.end();
   ++node_edges_it) {
10     const std::map<Key, Edge<KeyType>>>& local_node_edges =
        node_edges_it->second;
11     for (typename std::map<Key,
        Edge<KeyType>>>::const_iterator local_node_edges_it =
        local_node_edges.begin(); local_node_edges_it !=
        local_node_edges.end(); ++local_node_edges_it) {
12         Key local_key = local_node_edges_it->first;
13         typename std::map<Key, StorageNode>::const_iterator
        node_it = nodes.find(local_key);
14         if (node_it != nodes.end()) {
15             result.insert(node_it->second);
16         }
17     }
18 }
19
20 return result;
```

В листинге 9 представлен метод получения рёбер между хранилищами, что позволяет несколько упростить дальнейшие вычисления.

Листинг 9: Метод получения рёбер в другое хранилище

```
1 std::set<Edge<KeyType>> get_all_edges_to_storage(int
   target_storage_id) const {
2     std::set<Edge<KeyType>> result;
```

```

3     typename std::map<int, std::map<Key, std::map<Key,
Edge<KeyType>>>>::const_iterator storage_it =
external_edges.find(target_storage_id);
4     if (storage_it == external_edges.end()) {
5         return std::set<Edge<KeyType>>();
6     } else {
7         const std::map<Key, std::map<Key, Edge<KeyType>>>&
external_nodes_map = storage_it->second;
8         for (typename std::map<Key, std::map<Key,
Edge<KeyType>>>::const_iterator node_edges_it =
external_nodes_map.begin(); node_edges_it !=
external_nodes_map.end(); ++node_edges_it) {
9             const std::map<Key, Edge<KeyType>>& node_edge_map =
node_edges_it->second;
10            for (typename std::map<Key,
Edge<KeyType>>::const_iterator edges_it =
node_edge_map.begin(); edges_it != node_edge_map.end();
++edges_it) {
11                result.insert(edges_it->second);
12            }
13        }
14    }
15    return result;
16 }

```

2.2.4 Класс оптимизатора (optimizer.hpp)

Класс ExternalStorageOptimizer является центральным компонентом реализации алгоритма Кернигана-Лина и отвечает за расчёт метрики улучшения g_v .

Структура класса оптимизатора

Листинг 10: Класс StorageOptimizer

```

1 template <typename KeyType>
2 class ExternalStorageOptimizer {
3 private:
4     IBus<KeyType>* bus;
5 public:
6     ExternalStorageOptimizer(IBus<KeyType>* _bus)
7         : bus(_bus) {}
8 }

```

Основной метод расчёта метрик

Метод `calculate_gvs()` демонстрирует практическую реализацию итерации Boundary KL (алгоритм работы только с граничными вершинами). Он получает граничные вершины из обоих хранилищ и вычисляет для каждой из них метрику улучшения g_v , а метод `calculate_gv` занимается непосредственно расчётом метрики.

Листинг 11: Методы расчёта g_v

```
1 private:
2 float calculate_gv(const Node<KeyType>& node,
   std::set<Edge<KeyType>> boundary_edges) const {
3     NodeKey<KeyType> this_key = node.get_key();
4     float internal_edges_weight = 0;
5     float external_edges_weight = 0;
6
7     std::map<NodeKey<KeyType>, Edge<KeyType>> boundary_edges_map;
8     for (typename std::set<Edge<KeyType>>::const_iterator edge_it
   = boundary_edges.begin();
9         edge_it != boundary_edges.end(); ++edge_it) {
10        NodeKey<KeyType> other = edge_it->get_other(this_key);
11        if (!(other == this_key)) {
12            boundary_edges_map[other] = *edge_it;
13        }
14    }
15
16    for (typename std::map<NodeKey<KeyType>,
   Edge<KeyType>>::const_iterator edge_it = node.edges.begin();
17        edge_it != node.edges.end(); ++edge_it) {
18        const NodeKey<KeyType>& neighbor_key = edge_it->first;
19        const Edge<KeyType>& edge = edge_it->second;
20        if (boundary_edges_map.find(neighbor_key) !=
   boundary_edges_map.end()) {
21            external_edges_weight += edge.get_weight();
22        } else {
23            internal_edges_weight += edge.get_weight();
24        }
25    }
26    return internal_edges_weight - external_edges_weight;
```

```

27 }
28
29 public:
30 std::map<int, std::map<Node<KeyType>, float>> calculate_gvs(int
    storage1, int storage2) const {
31     std::map<int, std::map<Node<KeyType>, float>> result;
32     // Получаем граничные вершины для обоих хранилищ
33     std::set<Node<KeyType>> boundary_nodes1 =
bus->ask_neighbours_to_storage(storage1, storage2);
34     std::set<Node<KeyType>> boundary_nodes2 =
bus->ask_neighbours_to_storage(storage2, storage1);
35
36     std::set<Edge<KeyType>> boundary_edges1 =
bus->ask_edges_to_storage(storage1, storage2);
37     std::set<Edge<KeyType>> boundary_edges2 =
bus->ask_edges_to_storage(storage2, storage1);
38
39     result[storage1] = std::map<Node<KeyType>, float>();
40     typename std::set<Node<KeyType>>::const_iterator it;
41     for (it = boundary_nodes1.begin(); it !=
boundary_nodes1.end(); ++it) {
42         const Node<KeyType>& node = *it;
43         result[storage1][node.get_key()] = calculate_gv(node,
boundary_edges1);
44     }
45
46     result[storage2] = std::map<Node<KeyType>, float>();
47     for (it = boundary_nodes2.begin(); it !=
boundary_nodes2.end(); ++it) {
48         const Node<KeyType>& node = *it;
49         result[storage2][node.get_key()] = calculate_gv(node,
boundary_edges2);
50     }
51     return result;
52 };

```

Метод оптимизации

Метод `optimize()` демонстрирует практическую реализацию оптимизации KL. Он итеративно пользуется алгоритмом Кернигана-Лина, получает вершины с негативным g_v , после чего перемещает их в другое хранилище.

Листинг 12: Метод оптимизации

```

1 void optimize(int storage1, int storage2, int iterations_limit =
    5) {
2     if (iterations_limit == 0) return;
3
4     int iteration = 0;
5     std::map<int, std::set<Node<KeyType>>> negative_gvs =
        get_negative_gvs(calculate_gvs(storage1, storage2));
6     do {
7         typename std::map<int, std::set<Node<KeyType>>>::iterator
            map_it;
8         for (map_it = negative_gvs.begin(); map_it !=
            negative_gvs.end(); ++map_it) {
9             int this_storage = map_it->first;
10            int other_storage = this_storage == storage1 ?
                storage2 : storage1;
11            std::set<Node<KeyType>>& nodes = map_it->second;
12            typename std::set<Node<KeyType>>::const_iterator
                set_it;
13            for (set_it = nodes.begin(); set_it != nodes.end();
                ++set_it) {
14                const Node<KeyType>& node = *set_it;
15                Node<int> removed =
                    bus->request_node(node.get_key());
16                bus->send_remove_node(removed.get_key(),
                    this_storage);
17                bus->send_add_node(removed, other_storage);
18            }
19        }
20        ++iteration;
21        negative_gvs = get_negative_gvs(calculate_gvs(storage1,
            storage2));
22    } while (iteration < iterations_limit &&
        !negative_gvs.empty());
23 }

```

2.3 Streaming Graph Partitioning

Традиционные методы оптимизации распределения, такие как METIS, требуют полного знания структуры графа для выполнения разбиения и работают

в пакетном (offline) режиме. Однако во многих современных приложениях графы динамически изменяются: новые вершины и рёбра поступают непрерывно. Для таких сценариев разработаны методы потокового распределения (online partitioning).

В модели потокового распределения [8] вершины графа поступают на вход системы последовательно (в виде потока). Алгоритм должен принять решение о размещении каждой новой вершины v в одном из k шардов немедленно, основываясь только на информации об уже распределённых вершинах и, возможно, на знании соседей $N(v)$ новой вершины. Это накладывает жесткие ограничения на вычислительную сложность, но позволяет обрабатывать графы произвольного размера.

В работе [8] рассматривается семейство эвристик для потокового партиционирования, которые используют весовую функцию для учета текущей загруженности шардов и структуры связей. Цель эвристик - максимизировать количество соседей новой вершины в том шарде, куда она помещается. Общая формула для выбора шарда i выглядит следующим образом:

$$\text{ind} = \arg \max_{i \in \{1..k\}} \{|P^t(i) \cap N(v)| \cdot w(t, i)\}$$

где $P^t(i)$ - множество вершин в шарде i в момент времени t , а $w(t, i)$ - весовая функция, отвечающая за балансировку (например, линейная $w(t, i) = 1 - |P^t(i)|/C$ или экспоненциальная, где C - максимальная вместимость шарда). Одной из наиболее эффективных эвристик, согласно экспериментальным данным, является линейная детерминированная жадная эвристика (LDG).

2.4 Алгоритм Fennel

Алгоритм Fennel [10] представляет собой развитие идей потокового распределения графов и предлагает более формализованный подход к построению эвристики. Вместо комбинирования дискретных метрик (количество соседей) и весовых функций, Fennel вводит непрерывную объективную функцию $g(P)$, характеризующую качество текущего распределения P .

2.4.1 Объективная функция

Авторы Fennel определяют объективную функцию для k -распределения следующим образом:

$$g(P) = \sum_{i=1}^k [e(S_i, S_i) - p \binom{|S_i|}{2}]$$

где:

- $e(S_i, S_i)$ - количество рёбер, оба конца которых лежат внутри шарда S_i (внутренние рёбра).
- $p = 2\alpha = m \frac{k^{\gamma-1}}{n^{\gamma}}$, $1 \leq \gamma \leq 2$.
- α - параметр, регулирующий важность балансировки.
- γ - параметр, определяющий форму штрафа за размер шарда. В оригинальной работе рекомендуется $\gamma = \frac{3}{2}$.

Логика функции проста: она поощряет размещение рёбер внутри одного шарда (увеличение $e(S_i, S_i)$) и штрафует за слишком большой размер шарда (член $-p \binom{|S_i|}{2}$), что способствует балансировке.

2.4.2 Инкрементальное вычисление и формула для δg

Ключевым преимуществом Fennel для потоковой обработки является то, что для принятия решения о размещении новой вершины v не нужно пересчитывать $g(P)$ целиком. Достаточно вычислить прирост $\delta g(v, S_i)$, который получит система, если вершина v будет добавлена в существующий шард S_i .

Если предположить, что соседи v уже распределены, то добавление v в S_i увеличит количество внутренних рёбер в этом шарде на число соседей v , уже находящихся в S_i , то есть на $|S_i \cap N(v)|$.

Штраф за размер шарда изменяется с $-p \binom{|S_i|}{2}$ на $-p \binom{|S_i|+1}{2}$. Таким образом, изменение $\delta g(v, S_i)$ для шарда S_i вычисляется по формуле:

$$\delta g(v, S_i) = |S_i \cap N(v)| - p \left(\binom{|S_i|+1}{2} - \binom{|S_i|}{2} \right) \quad (1)$$

Именно это значение и используется в качестве основной эвристики. Алгоритм выбирает для вершины v тот шард i , который максимизирует $\delta g(v, S_i)$. Такой подход позволяет производить все вычисления распределенно: каждый шард может независимо рассчитать δg для себя, после чего выбирается шард с

максимальным значением, либо случайный среди максимальных равных.

В рамках данной курсовой работы был реализован модуль потокового распределения вершин и модифицированы реализованные ранее модули:

1. Классы имитации шины для общения компонентов
2. Класс Fennel-хранилища (FennelStorage), расширяющий обычное хранилище для поддержки алгоритма феннеля.

2.4.3 Структура проекта и основные зависимости

Языком разработки был выбран C++ в силу его низкоуровневости и скорости, а также с намерением в дальнейшем внедрить эти наработки в графовую БД на C++ научного руководителя.

Проект организован в виде набора заголовочных файлов (header files), что соответствует современным подходам разработки на C++. Основные файлы проекта:

- `graph.hpp` – содержит базовые классы для представления графовых структур.
- `interface_bus.hpp` – содержит интерфейс, описывающий основные сообщения в шине.
- `bus.hpp` – содержит простую реализацию имитации шины.
- `storage.hpp` – реализует класс хранилища для управления вершинами.
- `optimizer.hpp` – содержит реализацию оптимизатора с вычислением метрики g_v .
- `main.cpp` – демонстрационный файл с тестовым сценарием.

Ниже представлен релевантный для решения задачи практики код. Полный код представлен в приложении А.

2.4.4 Классы для представления графовых структур (`graph.hpp`)

Класс интерфейса шины (`interface_bus.hpp`)

Крайне важный интерфейс, через который происходит взаимодействие всей системы. Описывает методы добавления, удаления и запроса вершин, а также объявлений о добавлении и удалении для возможности другим хранилищам знать где находится другая вершина.

И наконец ключевые для алгоритма Кернигана-Лина методы получения граничных вершин и рёбер `ask_neighbours_to_storage` и `ask_edges_to_storage`.

Листинг 13: Базовая структура класса `IBus`

```
1 template <typename KeyType>
2 class IBus {
3 public:
4     virtual Node<KeyType> request_node(const NodeKey<KeyType>&
        node) = 0;
5     virtual int send_add_node(const Node<KeyType>& node) = 0;
6     virtual bool send_add_node(const Node<KeyType>& node, int
        storage_id) = 0;
7     virtual bool send_remove_node(const NodeKey<KeyType>& node) =
        0;
8     virtual bool send_remove_node(const NodeKey<KeyType>& node,
        int storage_id) = 0;
9     virtual int ask_who_has(int asker_id, NodeKey<KeyType> key) =
        0;
10    virtual void announce_add(NodeKey<KeyType> key, int
        storage_id, std::set<Edge<KeyType>> edges) = 0;
11    virtual void announce_remove(NodeKey<KeyType> key, int
        storage_id) = 0;
12    virtual float get_streaming_euristics_change(const
        Node<KeyType>& node, int storage_id) = 0;
13    // запрашивает у source вершины, соседствующие с target
14    virtual std::set<Node<KeyType>> ask_neighbours_to_storage(int
        source, int target) = 0;
15    // запрашивает у source рёбра, идущие в target
16    virtual std::set<Edge<KeyType>> ask_edges_to_storage(int
        source, int target) = 0;
17 };
```

Класс шины `SimpleBus` (`bus.hpp`)

Простая синхронная однопоточная реализация методов `IBus`. В данной работе важна реализация `send_add_node`, реализующий выбор хранилища для отправки вершины и вспомогательном метода `get_streaming_euristics_change`:

Листинг 14: Реализация метода `get_streaming_euristics_change`

```
1 int send_add_node(const Node<KeyType>& node) override {
2     std::vector<IStorage<KeyType>*> best_storages;
3     float best_euristics = -10000000;
4 }
```

```

5     for (typename std::map<int, IStorage<KeyType>*>::iterator it
      = storages.begin(); it != storages.end(); ++it) {
6         float this_euristics =
          it->second->get_streaming_euristics_change(node, total_edges);
7         if (this_euristics > best_euristics) {
8             best_storages.clear();
9             best_storages.push_back(it->second);
10            best_euristics = this_euristics;
11        } else if (this_euristics == best_euristics) {
12            best_storages.push_back(it->second);
13        }
14    }
15    if (best_storages.empty()) {
16        return -1;
17    } else {
18        std::uniform_int_distribution<> dist{0,
          best_storages.size() - 1};
19        IStorage<KeyType>* random_storage =
          best_storages[dist(gen)];
20        send_add_node(node, random_storage->get_id());
21        return random_storage->get_id();
22    }
23 };
24
25 float get_streaming_euristics_change(const Node<KeyType>& node,
      int storage_id) override {
26     if (storages.find(storage_id) == storages.end()) {
27         return false;
28     }
29     return
        storages[storage_id]->get_streaming_euristics_change(node,
        total_edges);
30 }

```

Как можно увидеть, метод опрашивает все хранилища и выбирает наилучшее. Если "лучших" окажется несколько (обычно если ни в одном хранилище нет соседей новой вершины), то выбирается случайный.

Класс хранилища (storage.hpp)

Класс Storage представляет собой хранилище вершин графа и реализует логику управления внутренними и внешними связями.

2.4.5 Структура класса FennelStorage

Для работы Феннель-алгоритму требуется знания о кол-ве шардов, а также балансировочное число. Для передачи этих данных используется структура FennelParameters, представленная в листинге 15.

Листинг 15: Структура FennelParameters

```
1 struct FennelParameters {
2     /***/int storage_number;
3     /**gamma*/float balancing_number;
4 };
```

Само же хранилище FennelStorage представлено в листинге 16

Листинг 16: Базовая структура класса Storage

```
1 template <typename KeyType>
2 class FennelStorage : public Storage<KeyType> {
3 protected:
4 typedef Node<KeyType> StorageNode;
5 typedef NodeKey<KeyType> Key;
6 FennelParameters params;
7 int number_of_nodes;
8 float streaming_euristics = 0;
9
10 float get_p_parameter(int number_of_nodes, long number_of_edges){
11     if (number_of_nodes == 0) {
12         return 0;
13     }
14     return 2 * number_of_edges * std::pow(params.storage_number ,
15         (params.balancing_number - 1)) / std::pow(number_of_nodes ,
16         params.balancing_number);
17 }
18
19 float fennel_function(const Node<KeyType>& node, long
20     total_edges) {
21     float p = get_p_parameter(number_of_nodes, total_edges);
22     float dg = 0;
23     for (typename std::map<Key, Edge<KeyType>>::const_iterator
24         edge_it = node.edges.begin(); edge_it != node.edges.end();
25         ++edge_it) {
26         const Key& neighbor_key = edge_it->first;
27         const Edge<KeyType>& edge = edge_it->second;
```

```

23         // Ищем соседа в текущем хранилище
24         typename std::map<Key, StorageNode>::iterator it2 =
        this->nodes.find(neighbor_key);
25         if (it2 != this->nodes.end()) {
26             ++dg;
27         }
28     }
29     dg -= p * this->size();
30     return dg;
31 }
32 public:
33
34 FennelStorage(int id, FennelParameters _params, std::map<Key,
        StorageNode> _nodes = std::map<Key, StorageNode>())
35     : Storage<KeyType>(id, _nodes), params(_params) {}
36
37 float get_streaming_euristics_change(const Node<KeyType>& node,
        long total_edges) override {
38     return fennel_function(node, total_edges);
39 }
40
41 void get_add_announcement(Key key, int announcer_id,
        std::set<Edge<KeyType>> edges) override {
42     ++number_of_nodes;
43     Storage<KeyType>::get_add_announcement(key, announcer_id,
        edges);
44 };
45
46 void get_remove_announcement(Key key, int announcer_id) override {
47     --number_of_nodes;
48     Storage<KeyType>::get_remove_announcement(key, announcer_id);
49 }
50 };

```

Заключение

не пустое

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Karypis G., Kumar V.* A fast and high quality multilevel scheme for partitioning irregular graphs // *SIAM Journal on Scientific Computing*. — 1998. — Т. 20, № 1. — С. 359—392. — DOI: 10.1137/S1064827595287997.
2. *Firth H., Missier P.* TAPER: query-aware, partition-enhancement for large, heterogenous, graphs. — 2016. — arXiv: 1603.04626 [cs.DB]. — URL: <https://arxiv.org/abs/1603.04626>.
3. *S. C. C. J. E. Z. Y. M.* Schism: a Workload-Driven Approach to Database Replication and Partitioning // *PVLDB*. — 2010. — Окт. — С. 48—57. — DOI: 10.14778/1920841.1920853.
4. *Quamar A. Kumar A D. A.* SWORD: Scalable workload-aware data placement for transactional workloads // *ACM International Conference Proceeding Series*. — 2013. — Март. — С. 430—441. — DOI: 10.1145/2452376.2452427.
5. *Priyadarshi A. K. J.* WawPart: Workload-Aware Partitioning of Knowledge Graphs // *Advances and Trends in Artificial Intelligence. Artificial Intelligence Practices: 34th International Conference on Industrial, Engineering and Other Applications of Applied Intelligent Systems, IEA/AIE 2021, Kuala Lumpur, Malaysia, July 26–29, 2021, Proceedings, Part I*. — Kuala Lumpur, Malaysia : Springer-Verlag, 2021. — С. 383—395. — ISBN 978-3-030-79456-9. — DOI: 10.1007/978-3-030-79457-6_33. — URL: https://doi.org/10.1007/978-3-030-79457-6_33.
6. LIRA: A Learning-based Query-aware Partition Framework for Large-scale ANN Search / X. Zeng [и др.] // *Proceedings of the ACM on Web Conference 2025*. — ACM, 04.2025. — С. 2729—2741. — (WWW '25). — DOI: 10.1145/3696410.3714633. — URL: <http://dx.doi.org/10.1145/3696410.3714633>.
7. Репозиторий ArangoDB. — 2025. — [Обращение: (21.09.2025)]. <https://github.com/arangodb/arangodb>.
8. *Stanton I., Kliot G.* Streaming Graph Partitioning for Large Distributed Graphs. — 2012.
9. *Andersen R., Peres Y.* Finding sparse cuts locally using evolving sets. — 2009. — DOI: 10.1145/1536414.1536449.

10. Fennel: Streaming Graph Partitioning for Massive Scale Graphs / C. Tsourakakis [и др.]. — 2014. — DOI: 10.1145/2556195.2556213.

Приложение А

Не пустое